
CSE D 501

*Linear Regression and Intro to
Assessing ML Model Performance*

Taylor Kessler Faulkner

University of Washington

October 1, 2025



Today's Class

- Linear Regression Cont'd, Ordinary Least Squares, Gradient Descent
- Break
- Assessing Performance
- Break
- Bias & Variance, Demo

Intro to Linear Regression Recap

Example goal: predict how much a given house is worth

Given: a dataset of houses with **input** and **labels**

$$\{(x_i, y_i)\}_{i=1}^n \text{ where } x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$$


Big idea behind regression:

$$y \approx f(x)$$

$(x_1[1], x_1[2], x_1[3], y_1) = (950 \text{ sq ft}, 2 \text{ bedrooms}, 1 \text{ bathroom}, 650\text{k})$

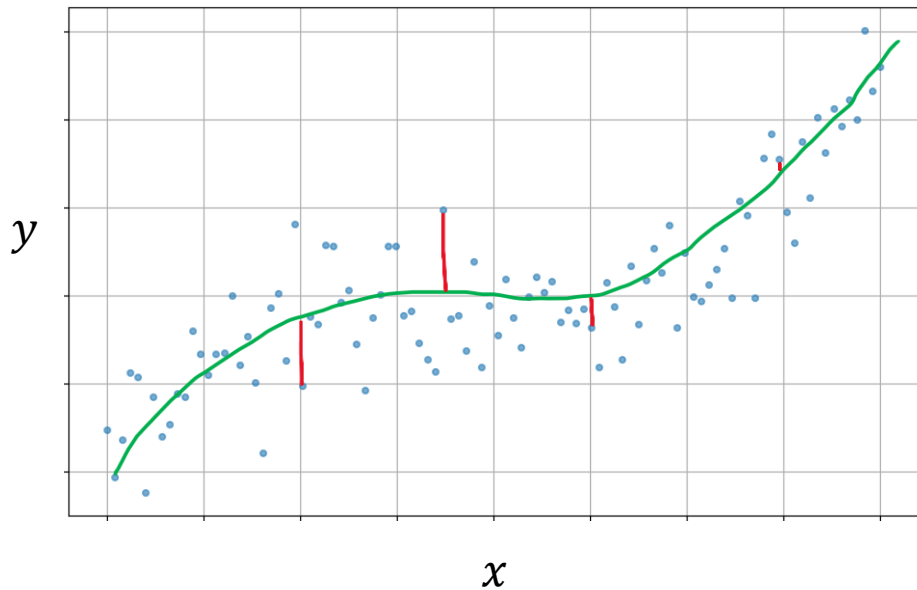
$(x_2[1], x_2[2], x_2[3], y_2) = (1200 \text{ sq ft}, 2 \text{ bedrooms}, 2 \text{ bathroom}, 950\text{k})$

$(x_3[1], x_3[2], x_3[3], y_3) = (800 \text{ sq ft}, 1 \text{ bedrooms}, 1 \text{ bathroom}, 550\text{k})$

...

$(x_n[1], x_n[2], x_n[3], y_n) = (1400 \text{ sq ft}, 3 \text{ bedrooms}, 2 \text{ bathroom}, 1100\text{k})$

Intro to Linear Regression Recap



Linear regression model: $y = f(x) + \epsilon$

$$\begin{aligned} &= \sum_{j=0}^D w_j h_j(x) + \epsilon \\ &= \boxed{w^T} \boxed{h(x)} + \boxed{\epsilon} \end{aligned}$$

Parameters/weights (green arrow pointing to w^T)

features (blue arrow pointing to $h(x)$)

noise (red arrow pointing to ϵ)

$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

$$E[\epsilon] = 0$$

Learned predictor: $\hat{y} = \hat{f}(x)$

Intro to Linear Regression Recap

How do we learn, $\hat{w} = [\hat{w}_0, \hat{w}_1, \dots, \hat{w}_D]$?

Each w defines a different function to explain the data: we want to pick the best one.

Definition of best?

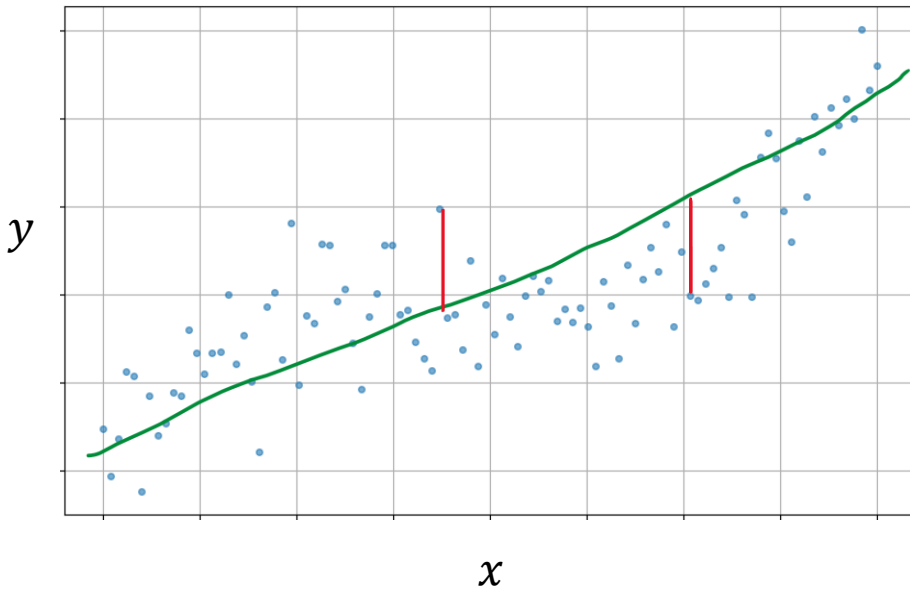
$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{Mean Squared Error (MSE)} = \frac{RSS}{n}$$

$$\text{Root Mean Squared Error (RMSE)} = \sqrt{MSE}$$

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Or RMSE, or other error metric



Linear/Polynomial Regression Assumptions

The data can be described with the following kind of function (additivity):

$$\begin{aligned} y &= f(x) + \epsilon \\ &= \sum_{j=0}^D w_j h_j(x) + \epsilon \end{aligned}$$

Normality of errors:

$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

Linear/Polynomial Regression Assumptions

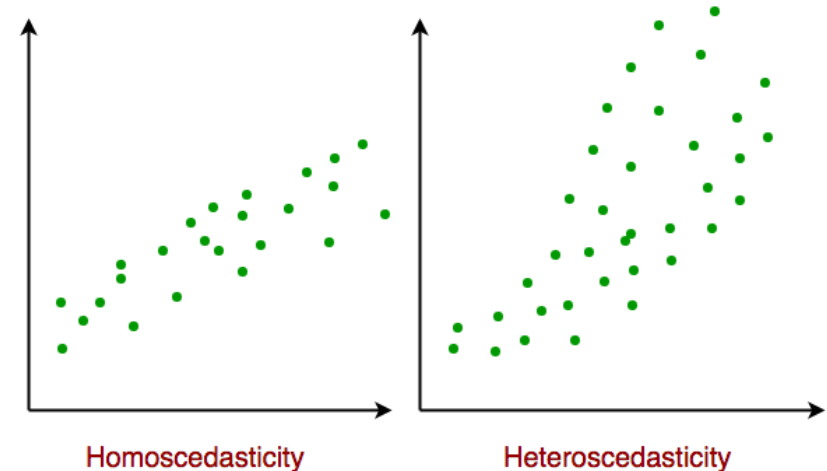
Independence: all observations (x_i, y_i) are **independent**

Example of dependence: measuring a child's growth over time where (x_i, y_i) are (date, height)

Homoscedasticity (constant variance of errors)

No Multicollinearity: Input variables should not be highly correlated with each other

Conversely, **input variables** being highly correlated with the **target value** is a good sign that regression will work well



Back to Finding Parameter Settings

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Ordinary Least squares: closed-form optimization function

Gradient descent: iterative algorithm to find the minimum of a cost function

Matrix Form

$$y_i = x_i^T w$$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \dots \\ y_n \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} x_1^T \\ \dots \\ x_n^T \end{bmatrix}$$

Matrix form

$$\mathbf{y} = \mathbf{X}w$$

Ordinary Least Squares

$$\hat{w}_{LS} = \underset{w}{\operatorname{argmin}} \sum_{i=1}^n (y_i - x_i^T w)^2$$

Residuals



Matrix Form


$$\begin{aligned}\hat{w}_{LS} &= \underset{w}{\operatorname{argmin}} \|y - Xw\|_2^2 \\ &= \underset{w}{\operatorname{argmin}} (y - Xw)^T (y - Xw)\end{aligned}$$

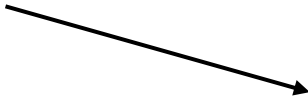
Aside: l_2 norm

$$\begin{aligned}l_2 \text{ norm: } \|z\|_2 &= \sqrt{\sum_{i=1}^n z_i^2} = \sqrt{z^T z} \\ \|z\|_2^2 &= \sqrt{z^T z}^2 = z^T z\end{aligned}$$

Ordinary Least Squares

You don't need to know how
to compute directional
derivatives for this class,
showing in case of interest

$$\begin{aligned}\widehat{w}_{LS} &= \underset{w}{\operatorname{argmin}} \|y - Xw\|_2^2 \\ &= \underset{w}{\operatorname{argmin}} (y - Xw)^T (y - Xw)\end{aligned}$$


$$\nabla_w \|y - Xw\|_2^2 \cdot \delta w = 2\delta w^T (X^T y - X^T Xw)$$

The minimum occurs when directional derivatives are zero

Normal equations:

$$X^T y = X^T Xw$$

Solving for w :

$$\widehat{w}_{LS} = (X^T X)^{-1} X^T y$$

Ordinary Least Squares Example

Two data points: (3, 5), (5,7)

Find a (intercept) and b (slope) for the ordinary least squares solution in matrix form for $\hat{y} = ax + b$

w vector in terms of a and b ?

X ?

y ?

$$X = \begin{bmatrix} x_1^T \\ \dots \\ x_n^T \end{bmatrix}$$

$$y = \begin{bmatrix} y_1 \\ \dots \\ y_n \end{bmatrix}$$

$$y = Xw$$

$$\widehat{w}_{LS} = (X^T X)^{-1} X^T y$$

Ordinary Least Squares Example

Two data points: (3, 5), (5, 7)

Find a (intercept) and b (slope) for the ordinary least squares solution in matrix form for $\hat{y} = ax + b$

$$\mathbf{y} = \begin{bmatrix} 5 \\ 7 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & 3 \\ 1 & 5 \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} b \\ a \end{bmatrix}$$

$$\mathbf{X} = \begin{bmatrix} x_1^T \\ \dots \\ x_n^T \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \dots \\ y_n \end{bmatrix} \quad \mathbf{y} = \mathbf{X}\mathbf{w} \quad \widehat{\mathbf{w}}_{LS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Ordinary Least Squares Example

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 1 & 5 \end{bmatrix} = \begin{bmatrix} 2 & 8 \\ 8 & 34 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} 1 & 1 \\ 3 & 5 \end{bmatrix} \begin{bmatrix} 5 \\ 7 \end{bmatrix} = \begin{bmatrix} 15 \\ 50 \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \begin{bmatrix} \frac{17}{2} & -2 \\ -2 & \frac{1}{2} \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$a = 1, b = 2$$

$$\hat{y} = 1x + 2$$

$$\hat{y}_1 = 3 + 2 = 5$$

$$\hat{y}_2 = 5 + 2 = 7$$

$$\mathbf{w} = \begin{bmatrix} b \\ a \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & 3 \\ 1 & 4 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$$

$$\hat{\mathbf{w}}_{LS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Back to Finding Parameter Settings

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Ordinary Least squares: closed-form optimization function

Gradient descent: iterative algorithm to find the minimum of a cost function

Gradient Descent

Convex function when parameters are linear

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Algorithm (pseudocode)

$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

$\nabla L(w)$: the gradients of loss function L
(ex: MSE) on a set of weights w

I.e., each w_i is updated as

$$w_i^{(t+1)} = w_i^{(t)} - \eta \frac{dL(w_i)}{dw_i}$$

Convergence: $w^{(t+1)} - w^{(t)} < \epsilon$

Gradient Descent requires **convex** function to converge to optimal solution

Gradient Descent

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Algorithm (pseudocode)

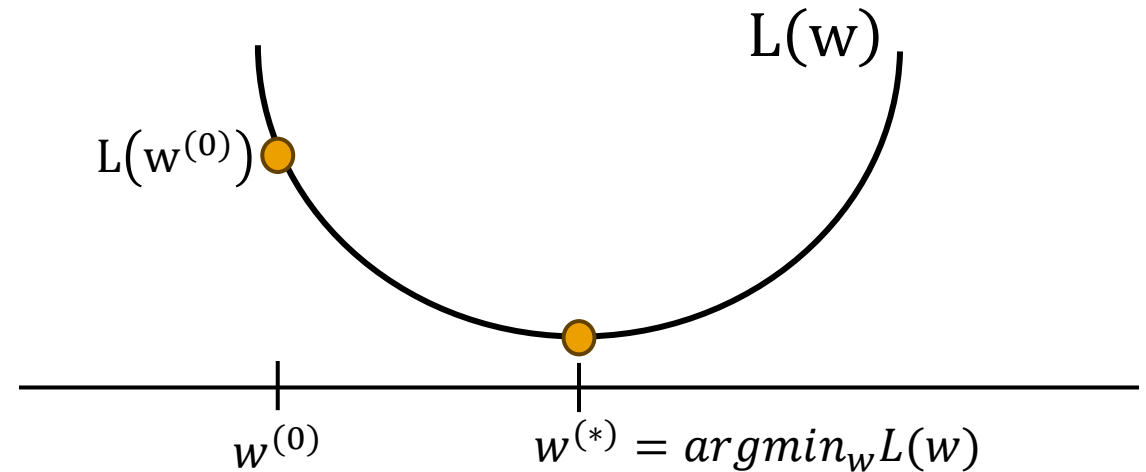
$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η



Gradient Descent

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Algorithm (pseudocode)

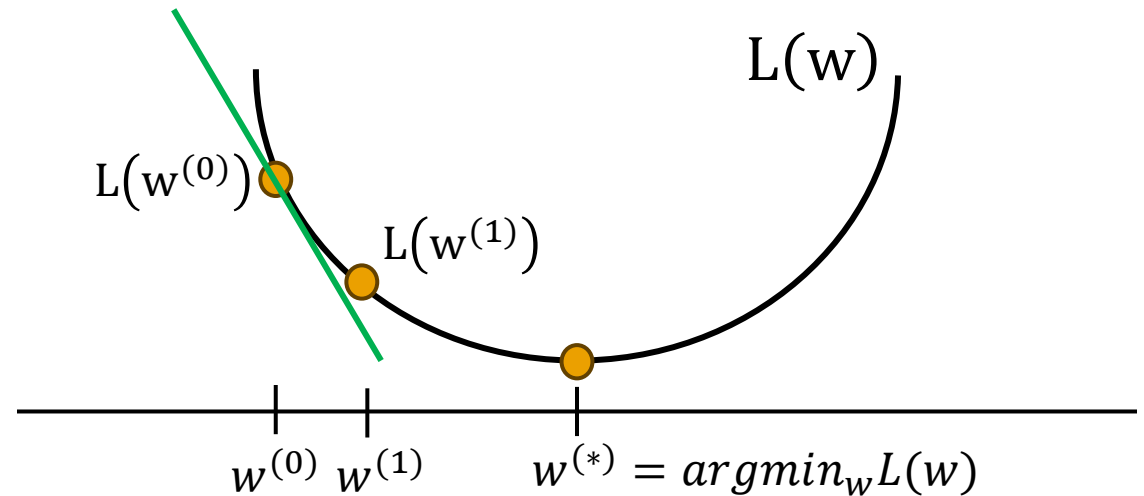
$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η



Gradient Descent

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

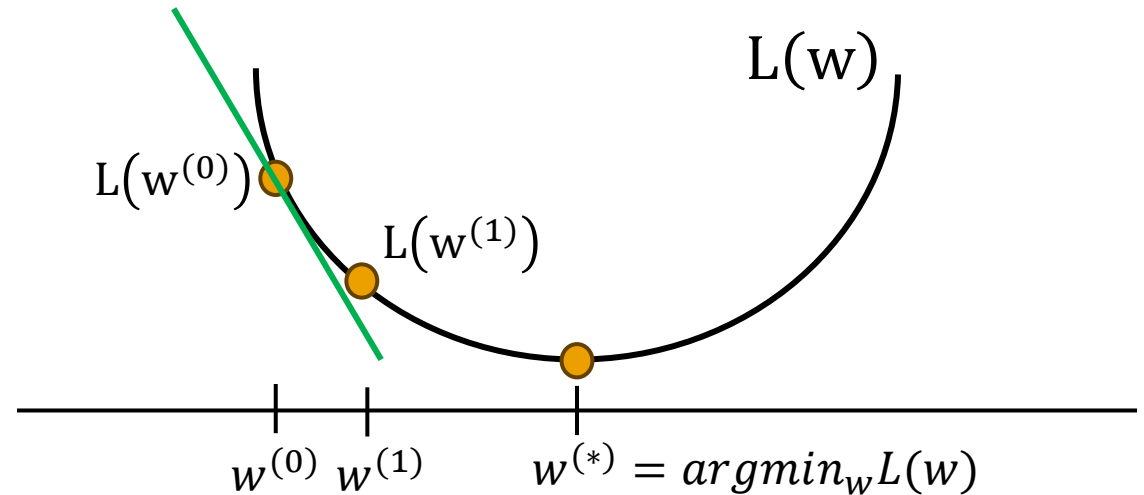
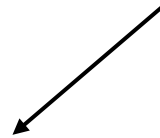
Algorithm (pseudocode)

$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

Gradient: vector of partial derivatives



Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

Gradient Descent

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Algorithm (pseudocode)

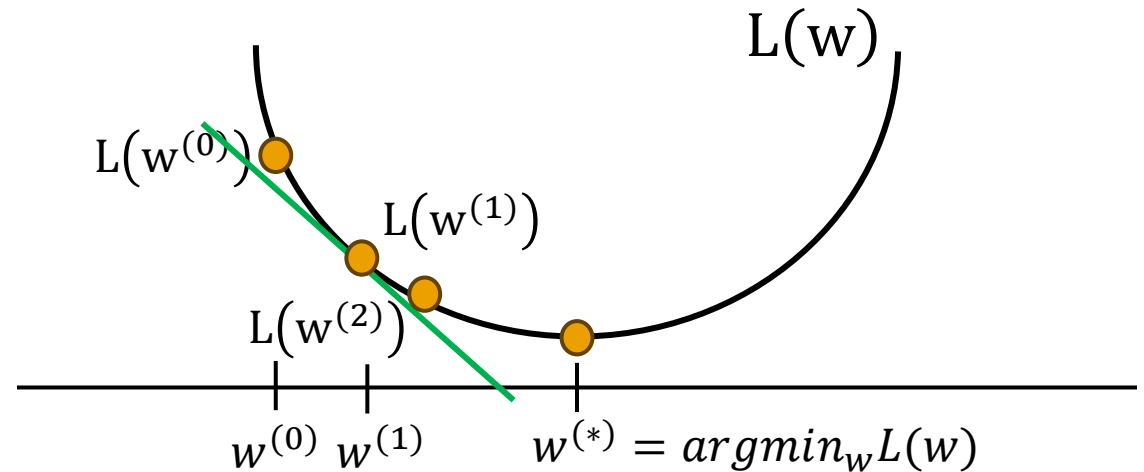
$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η



Gradient Descent

$$\hat{w} = \underset{w}{\operatorname{argmin}} MSE(w)$$

Algorithm (pseudocode)

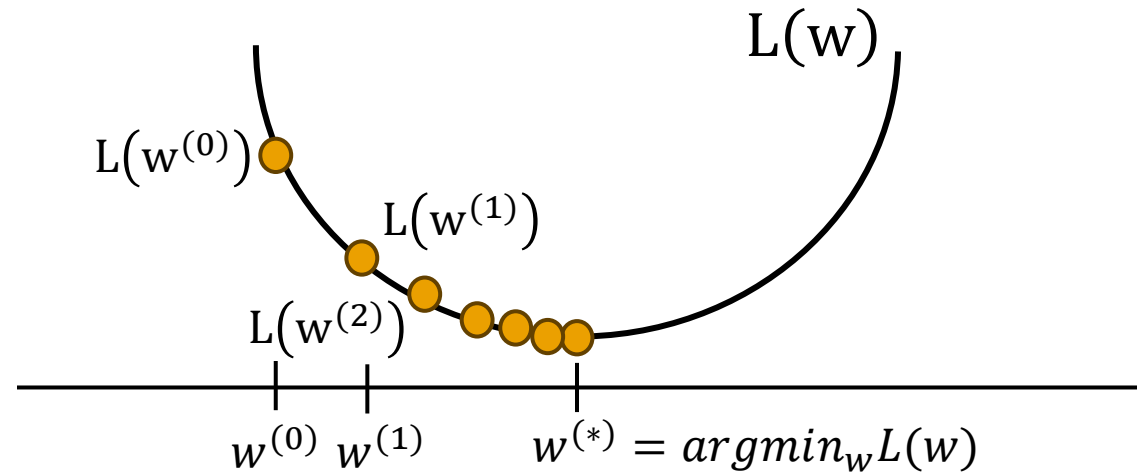
$t = 0$

while not converged:

$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w)$$

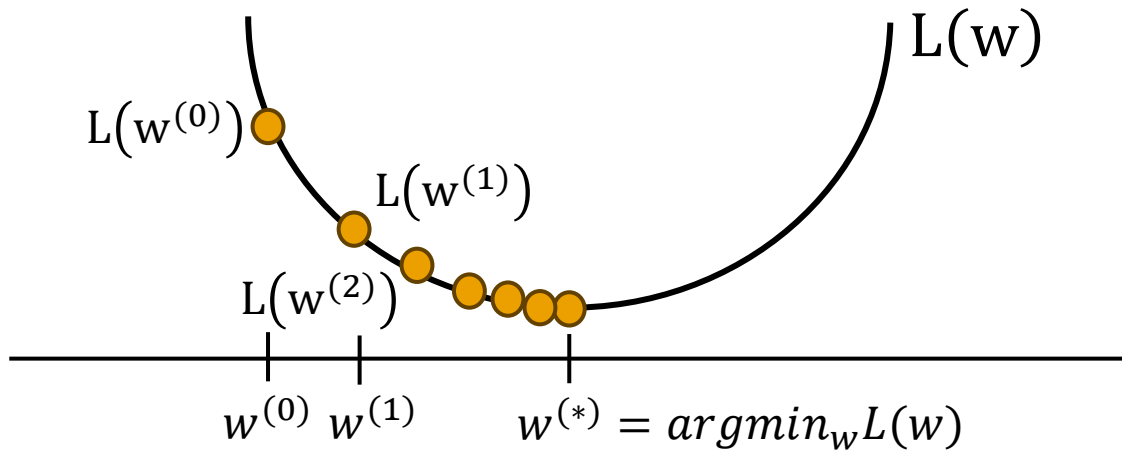
Hyperparameters:

- Initial point(s) $w^{(0)}$ (can be random)
- Step size η

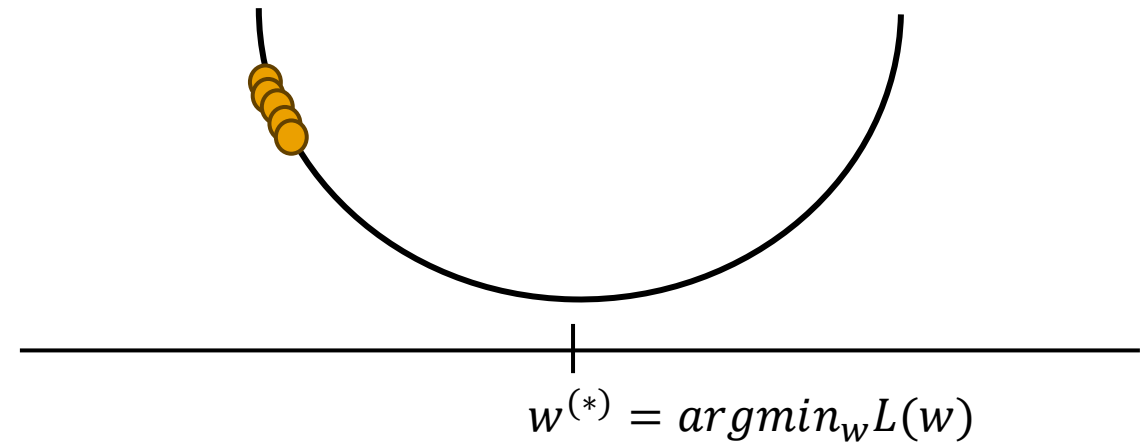


Gradient Descent: Choosing Step Size

Good Step Size



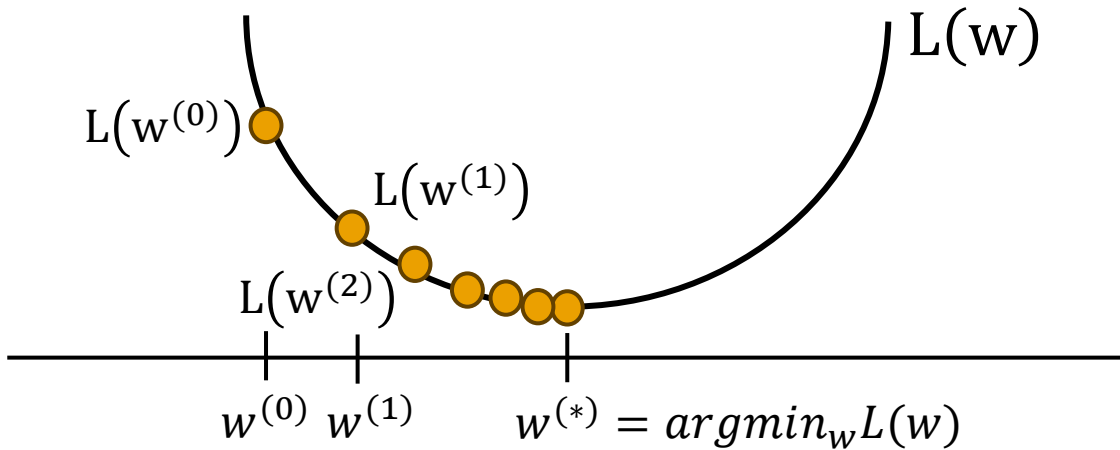
Step Size Too Small



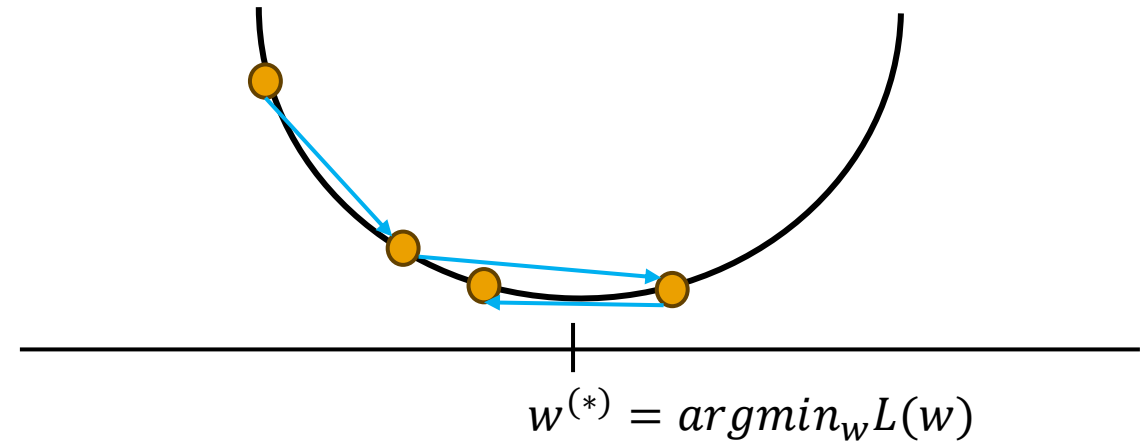
- Takes tiny steps
- Too long to converge

Gradient Descent: Choosing Step Size

Good Step Size



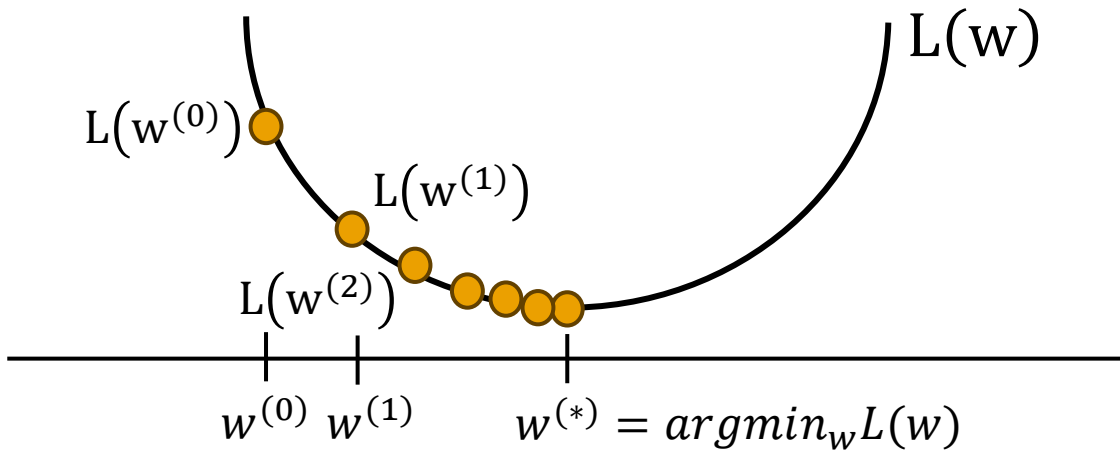
Step Size Too Large



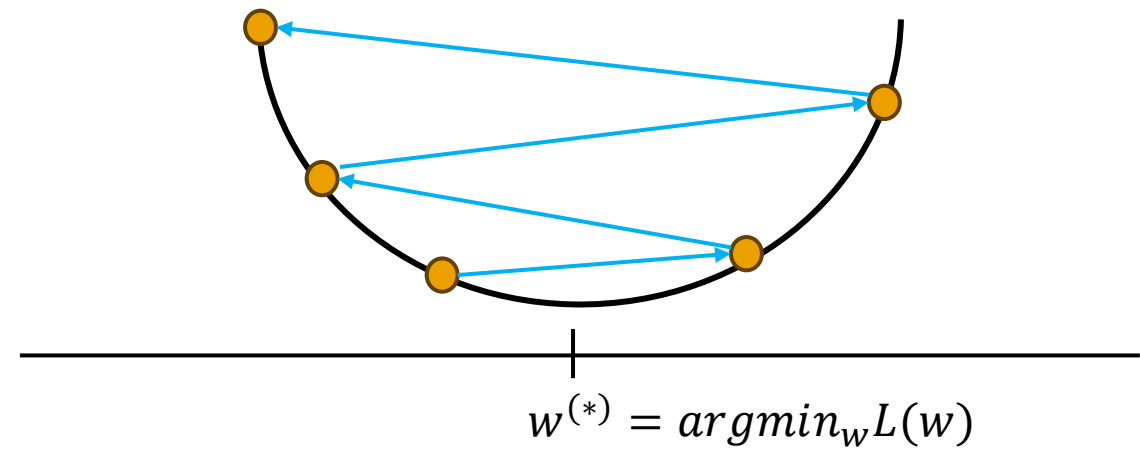
- Takes large steps
- Can skip over/oscillate over optimum

Gradient Descent: Choosing Step Size

Good Step Size



Step Size Too Large



- Takes large steps
- Can skip over/oscillate over optimum
- Can even diverge in worst case

Ordinary Least Squares vs Gradient Descent

Ordinary Least Squares

- Good for smaller datasets that fit in computer memory
- Solves for exact solution
- Uses entire dataset to solve, so used for **offline** learning

Offline learning: train on entire dataset at once, then deploy

Gradient Descent

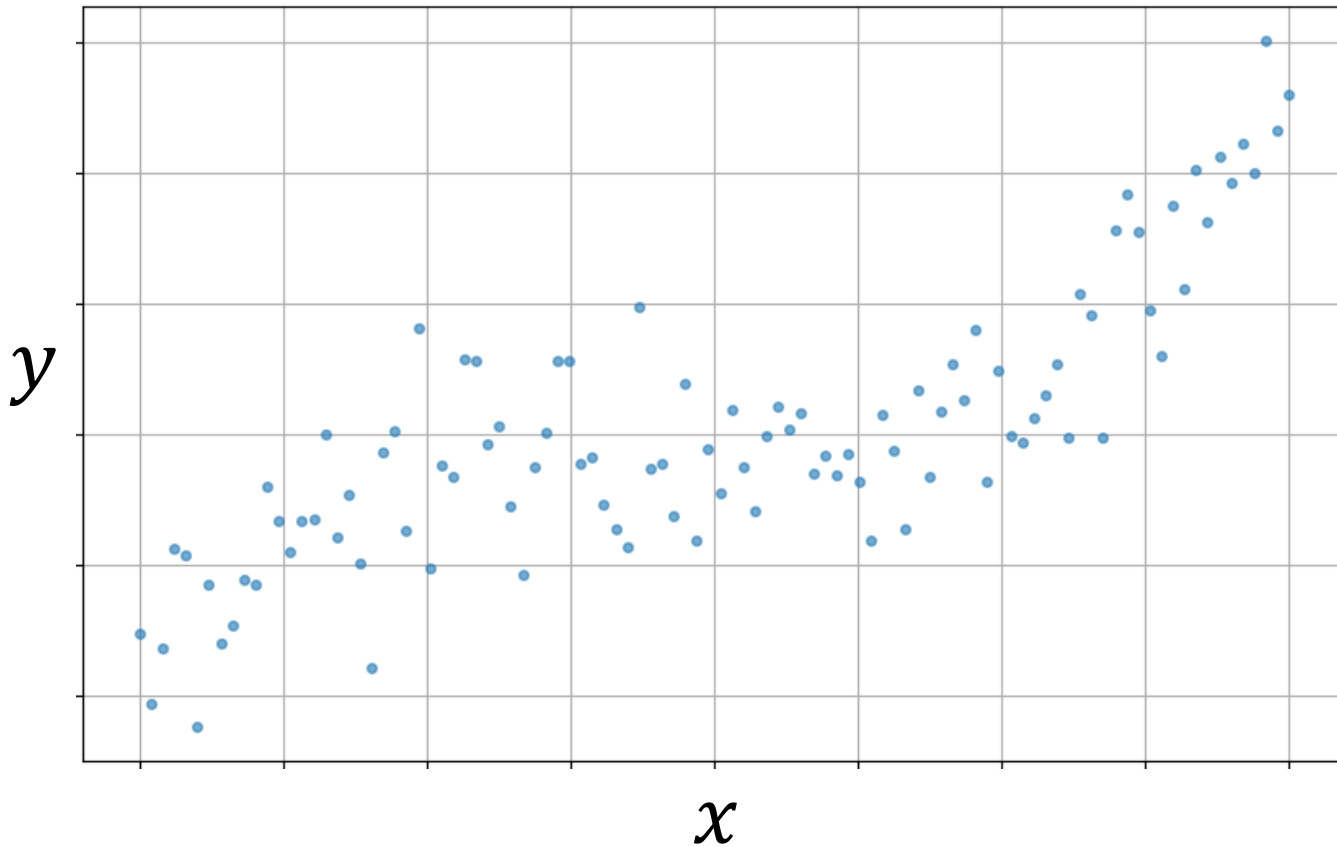
- Good for very large datasets that cannot fit in computer memory
- Gives approximate solution (within convergence tolerance)
- Does not require the entire dataset, can be trained **online**

Online learning: train in *batches* (subsets of the dataset), as data comes in. May be deployed while still learning

Break

Get some water, stretch your legs, chat, be back in 10

Applying Regression: complexity



Is minimizing loss on the training data the true goal?

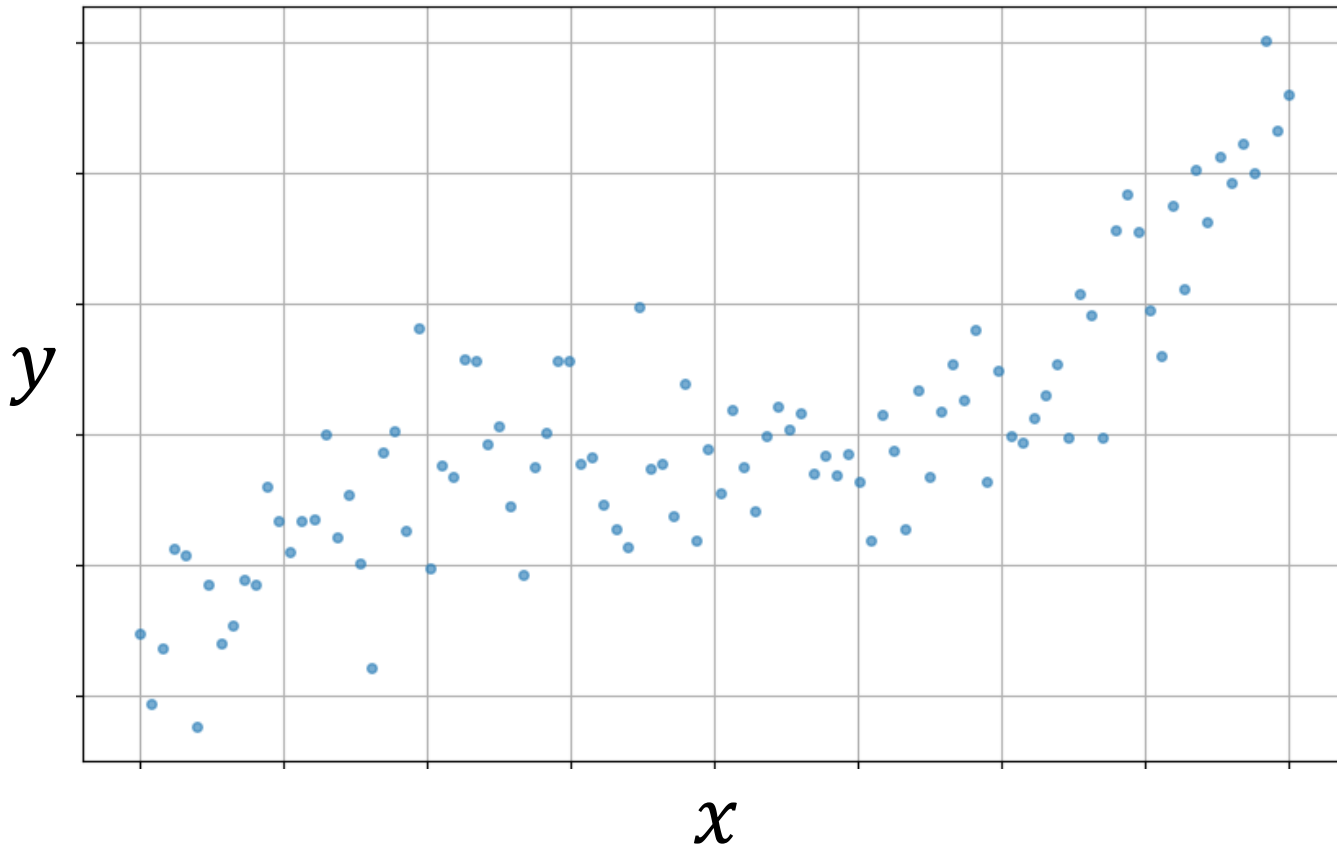
$$\hat{y}_i = w_0 + w_1 x_i$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 + w_3 x_i^3$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 + \cdots + w_n x_i^n$$

Applying Regression: complexity



$$\hat{y}_i = w_0 + w_1 x_i$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 + w_3 x_i^3$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 + \dots + w_n x_i^n$$

Which has the lowest MSE?

A: the n -degree polynomial

Which is the best for predicting future data?

A: the degree 3 polynomial

Applying Regression: performance

What is the point of training ML Models?

- Usually, we want them to **generalize**
 - Predict accurately on **unseen** data
 - Ex: You've trained on a dataset of houses, and someone asks you to predict what their house will sell for

Does minimizing MSE on all of our data give us the most generalizable model?

- No, this gives us data that is very well fit to the data we have, not new data

Applying Regression: performance

How do we measure performance on unseen data?

- The unattainable measure is our **true error**
- Consider the **probability density function** (PDF) of some input x and output y
 - Some probability associated with x (ex: probability that a house has 1200 sq ft)
 - Some probability associated with y given x (ex: probability that a house costs 1.2 mil if house is 1200 sq ft)
- Have a general loss function $L(y, \hat{f}(x))$ (could be MSE)
- True error:

$$\mathbb{E}_{(x,y)} \left[L(y, \hat{f}(x)) \right] = \sum_x \sum_y L(y, \hat{f}(x)) p(x, y)$$

- Unfortunately, we don't know the true probabilities

Applying Regression: performance

So how do we estimate performance on unseen data?

- Hold out data as a **test set** to estimate how well our model will perform

Steps: split data into **training set** and **testing set**

- **Training set:** training the model to learn parameters
- **Testing set:** testing the model to estimate the future performance on unseen data
- *Important notes:*
 - The train and test set should be **randomly** split from all of the data
 - If data is sorted or has a structure (timed data, sorted inputs, etc), splitting the data without randomly sorting will not create representative sets
 - **Never train on the test set or use to choose hyperparameters**
 - In practice, often set aside 10-20% of the data as a test set

Model Complexity

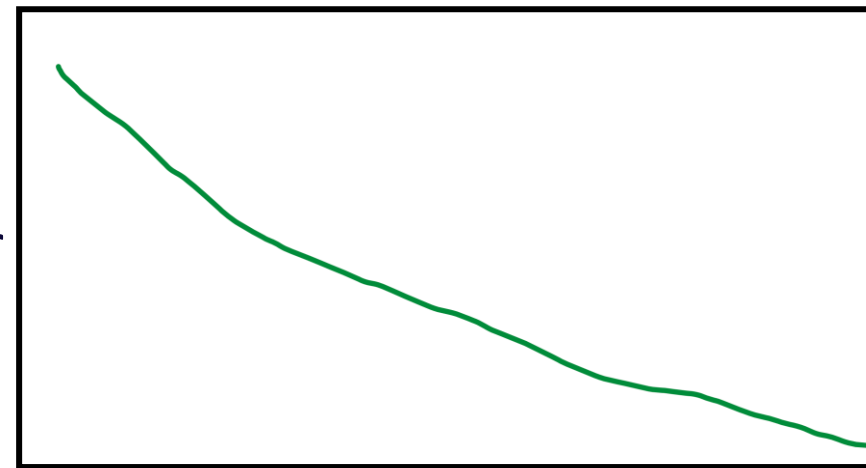
- Complexity is...complex
- Not fully defined, but in regression it is usually correlated with the number of parameters and input degrees

$$\hat{y}_i = w_0 + w_1 x_i$$

$$\hat{y}_i = w_0 + w_1 x_i + w_2 x_i^2 + \dots + w_n x_i^n \quad \longleftarrow \text{More complex}$$

- In general, higher complexity lowers our **training error**

Training error



Model complexity

Model Complexity

- How might model complexity affect **true error/test error**?

True error



Model complexity

- A: Depends on the complexity of the true function

Underfitting/Overfitting

- Underfitting: trained model does not capture the patterns in the training data
 - Too simple
- Overfitting: trained model is too closely fit to the training data to generalize well
 - Too complex
 - Learns “noise” in the data
- Mathematical definition of overfitting:
 - Given a trained predictor $\hat{w}$, there exists another predictor w' from the same model class s.t.

$$\begin{aligned} error_{true}(w') &< error_{true}(\hat{w}) \\ error_{train}(w') &> error_{train}(\hat{w}) \end{aligned}$$

Applying Regression: performance

How to choose hyperparameters:

- Use a **validation set**
- Use **cross validation**

Validation Set

If we can't choose hyperparameters based on the training or testing sets:



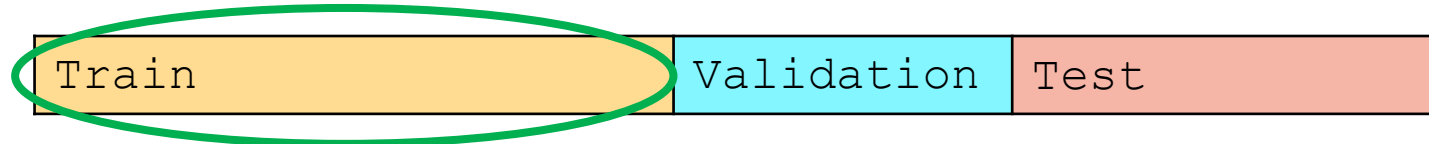
We can split the data into a third dataset, the **validation set**



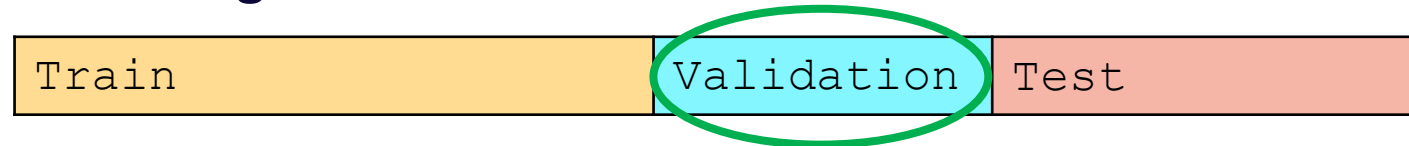
Validation Set

General process:

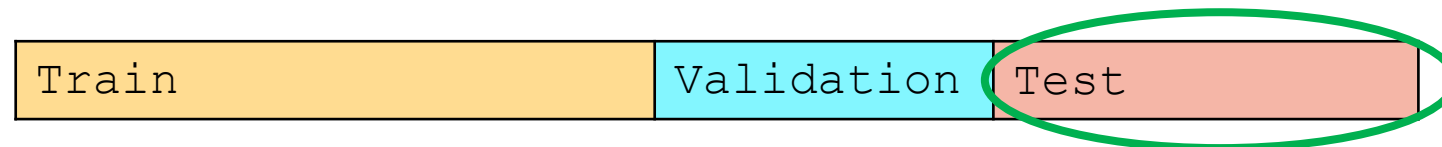
- For every hyperparameter setting you want to test:
 - Train your model on the **training set**



- Test the error metric on the **validation set**
- Keep track of the hyperparameter setting with the smallest error



- Choose the hyperparameter with the smallest error
- Report the expected error on unseen data as the error metric on the **test set**



Applying Regression: performance

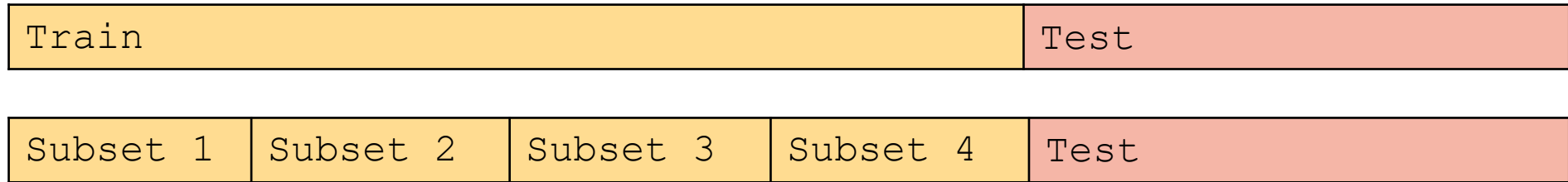
How to choose hyperparameters:

- Use a **validation set**
- Use **cross validation**

Cross-Validation

Because of the potential downsides, people often use cross-validation

Big idea: use subsets of the **training set** to act as a validation set



- For every hyperparameter, train your model k times
 - Use $k - 1$ subsets to train
 - Use 1 subset (different every time) as a **validation set**
 - Average the errors/performance over all subsets/validation sets

Cross-Validation

General process:

- For every hyperparameter setting you want to test:
 - For i in $[1, k]$
 - Train your model on the **training data – subset i**
 - Test the error on **subset i**
 - Average the error over all subsets
 - Keep track of the hyperparameter setting with the smallest average error
- Choose the hyperparameter with the smallest error
- Train model on **all subsets**
- Report the expected error on unseen data as the error metric on the **test set**

Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test
Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test

Cross-Validation

General process:

- For every hyperparameter setting you want to test:
 - For i in $[1, k]$
 - Train your model on the **training data – subset i**
 - Test the error on **subset i**
 - Average the error over all subsets
 - Keep track of the hyperparameter setting with the smallest average error
- Choose the hyperparameter with the smallest error
- Train model on **all subsets**
- Report the expected error on unseen data as the error metric on the **test set**

Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test
Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test

Cross-Validation

General process:

- For every hyperparameter setting you want to test:
 - For i in $[1, k]$
 - Train your model on the training data – subset i
 - Test the error on subset i
 - Average the error over all subsets
 - Keep track of the hyperparameter setting with the smallest average error
- Choose the hyperparameter with the smallest error
- Train model on **all subsets**
- Report the expected error on unseen data as the error metric on the **test set**

Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test
Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test

Cross-Validation

General process:

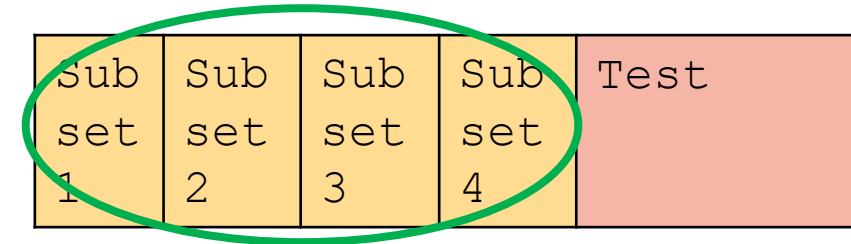
- For every hyperparameter setting you want to test:
 - For i in $[1, k]$
 - Train your model on the training data – subset i
 - Test the error on subset i
 - Average the error over all subsets
 - Keep track of the hyperparameter setting with the smallest average error
- Choose the hyperparameter with the smallest error
- Train model on **all subsets**
- Report the expected error on unseen data as the error metric on the **test set**

Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test
Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test

Cross-Validation

General process:

- For every hyperparameter setting you want to test:
 - For i in $[1, k]$
 - Train your model on the **training data – subset i**
 - Test the error on **subset i**
 - Average the error over all subsets
 - Keep track of the hyperparameter setting with the smallest average error
- Choose the hyperparameter with the smallest error
- **Train model on all subsets**
- Report the expected error on unseen data as the error metric on the **test set**



A diagram illustrating the data partitioning for cross-validation. It consists of a horizontal row of five colored boxes. The first four boxes are yellow and each contains the text 'Sub set' followed by a number from 1 to 4. The fifth box is red and contains the text 'Test'. A green oval is drawn around the four yellow boxes, indicating that these subsets are used for training the model in each iteration of the cross-validation process.

Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test
-----------------	-----------------	-----------------	-----------------	------

Cross-Validation

General process:

- For every hyperparameter setting you want to test:
 - For i in $[1, k]$
 - Train your model on the **training data – subset i**
 - Test the error on **subset i**
 - Average the error over all subsets
 - Keep track of the hyperparameter setting with the smallest average error
- Choose the hyperparameter with the smallest error
- Train model on **all subsets**
- Report the expected error on unseen data as the error metric on the **test set**

Sub set 1	Sub set 2	Sub set 3	Sub set 4	Test
-----------------	-----------------	-----------------	-----------------	------

Validation Set

Discussion: pros and cons of using a validation set?

Pros

- Easy to implement
- Fairly fast (for every hyperparameter, requires training a model and finding error on the validation set)

Cons

- Less training data to use
- Can overfit

Cross-Validation

Discussion: pros and cons of using cross-validation?

Pros

- Helps prevent overfitting to a single validation or training set
- You can use all of the training data

Cons

- Slow: for every hyperparameter settings, you train a model k times
 - Often $k = 4$ to 10
- Computationally expensive

Examples

- You want to predict the stock price of Amazon given historical stock price data
- You create a train set and a test set, where the training set is the daily stock price up to Jan 1, 2025 and the test set is the daily stock price from Jan 2, 2025 – Sep 1, 2025
- What is wrong with this procedure?

Examples

- You have a dataset with 100 inputs and n examples
- You decide to use the 5 inputs that have the highest correlation scores with the labels on the entire dataset
- After choosing those features, you split the data into a training and testing set
- You train linear regression using the selected features on the training set, then compute the error on the test set and report that as your expected error on unseen data
- Is anything wrong with this procedure?

Break

Get some water, stretch your legs, chat, be back in 10

Model Error Types

- Bias
- Variance
- Irreducible error
 - We can't control irreducible error: comes from uncontrollable aspects such as noisy observations
 - Example: data from a US census may be noisy based on self-reported information

Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

$$\mathbb{E}[\hat{f}(x)] - f(x)$$

- Can't compute the true bias; why?
- We don't have access to the true function $f(x)$! We can estimate by running a model M on subsets of the data, finding the expected predictions, and comparing to a test set

Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

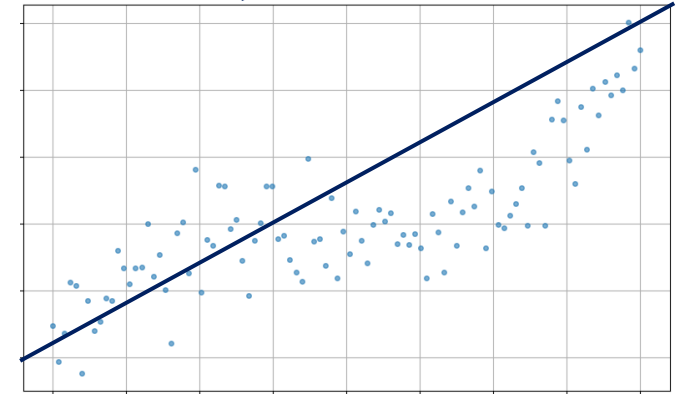
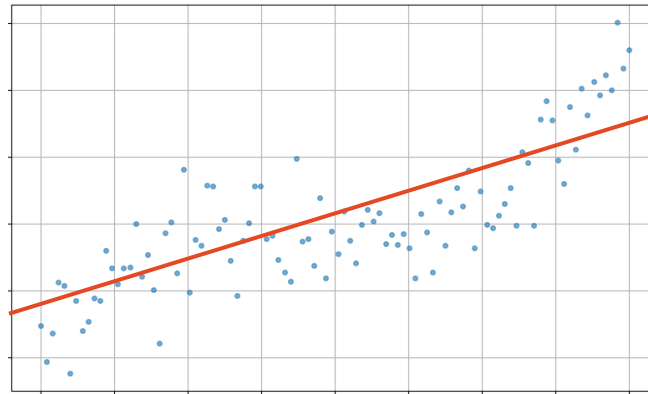
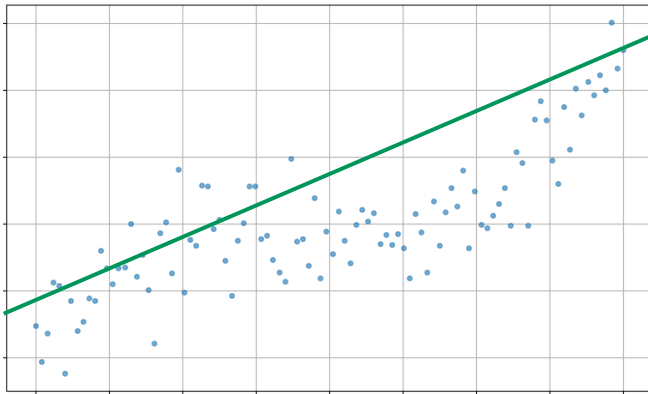
$$\mathbb{E}[\hat{f}(x)] - f(x)$$

- Less complex models tend to have higher bias, as they cannot capture the complexity/patterns in the data

Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

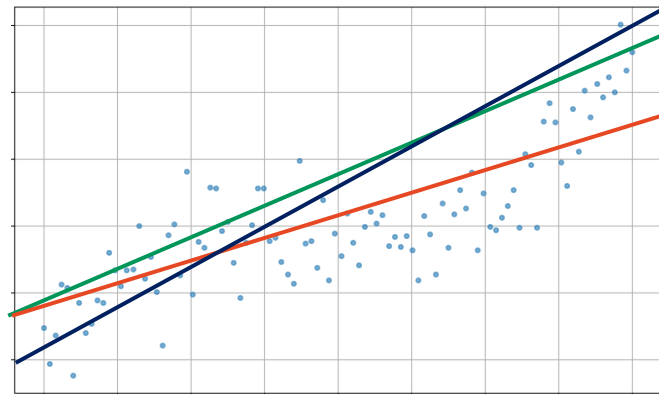
$$|\mathbb{E}[\hat{f}(x)] - f(x)|$$



Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

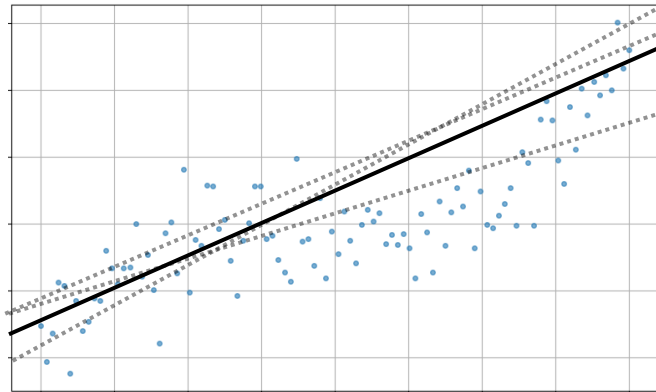
$$|\mathbb{E}[\hat{f}(x)] - f(x)|$$



Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

$$|\mathbb{E}[\hat{f}(x)] - f(x)|$$

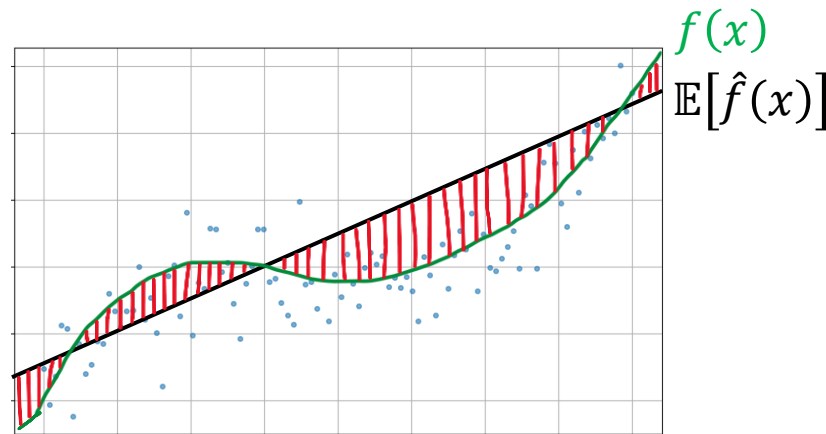


Bias

- Bias is the difference between the **expected** prediction of the trained model and the **actual value** it is attempting to predict

$$|\mathbb{E}[\hat{f}(x)] - f(x)|$$

Could estimate with test set



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

- Could test by running a model M on subsets of the data, finding the average of the predictions, and comparing that average to each individual prediction

Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

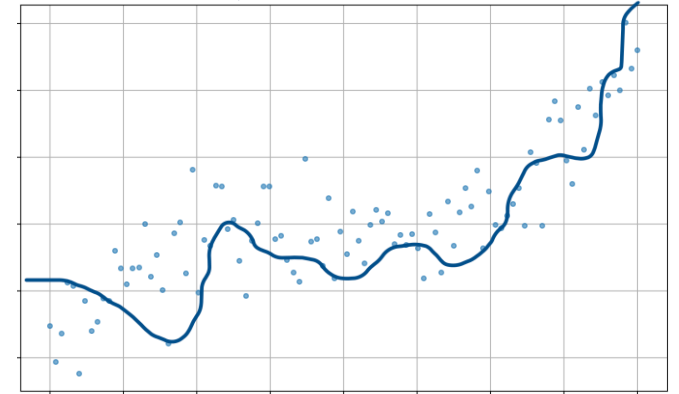
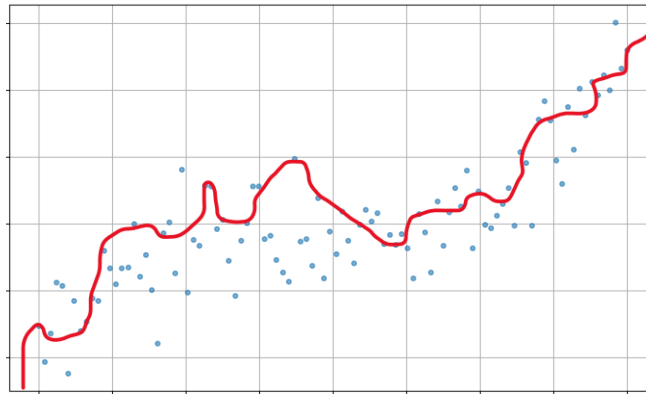
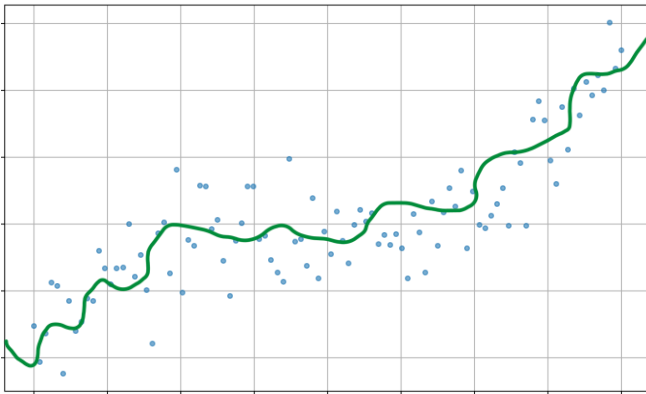
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

- More complex models tend to have higher variance, as they fit more closely to the training data, causing high variations in learned parameters

Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

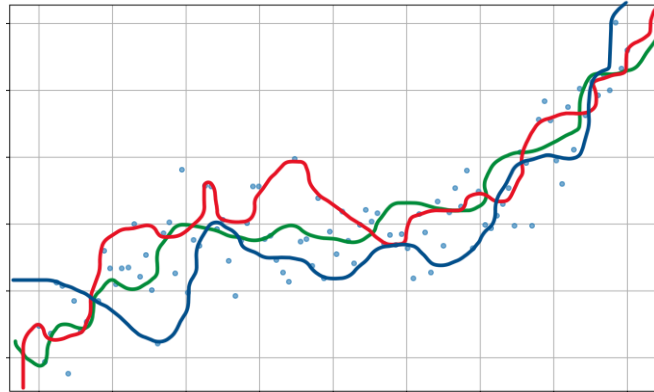
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

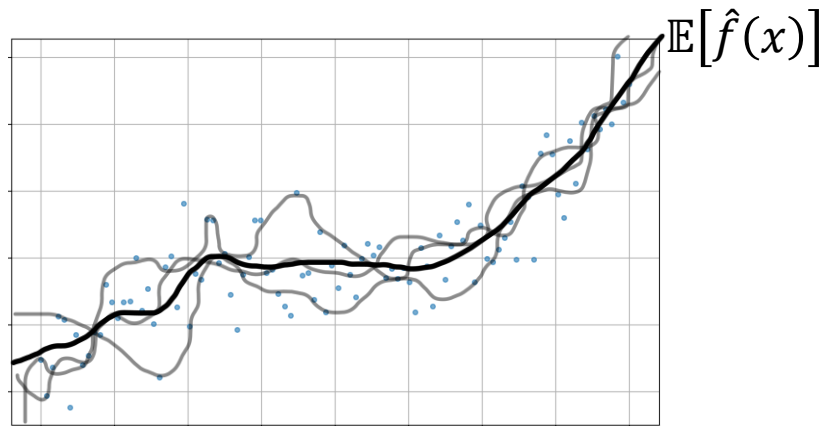
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

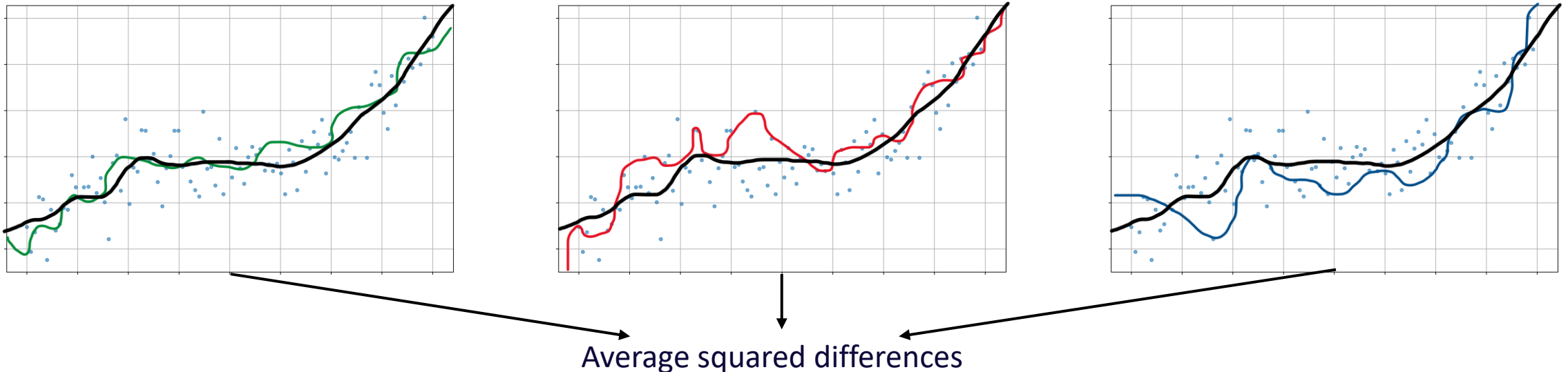
$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Variance

- Variance is the variability in the model prediction: if training on different data, how different are the predictions?

$$\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



Bias-Variance Tradeoff

- Simpler models tend to have **high bias** and **low variance**
- Complex models tend to have **low bias** and **high variance**

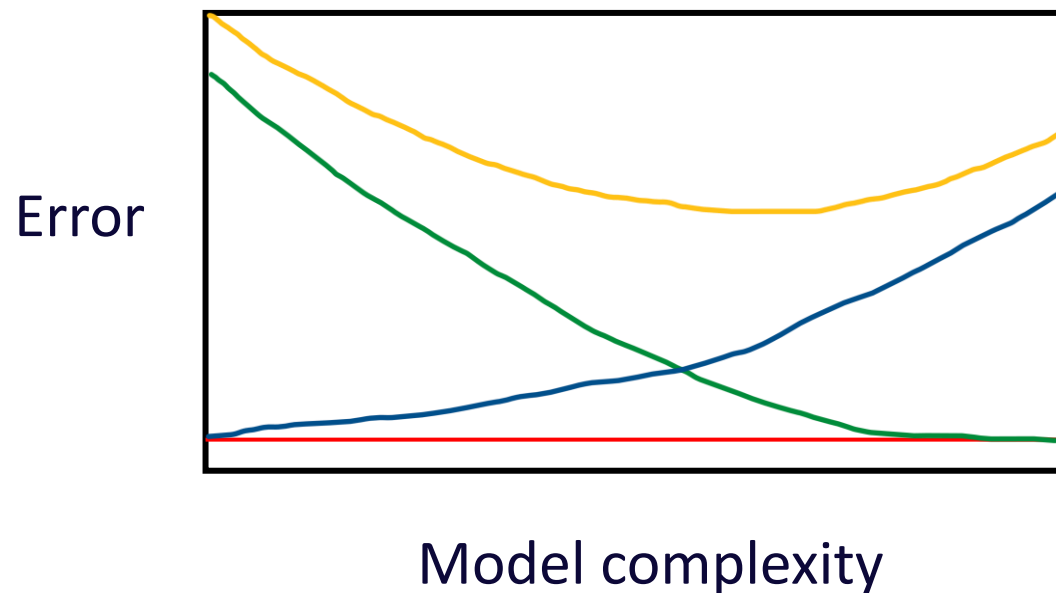
$$\mathbb{E} \left[\left(y - \hat{f}(x) \right)^2 \right] = \left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2 + \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right] + \sigma_{\epsilon}^2$$

Total error = bias squared + variance + irreducible error

Bias-Variance Tradeoff

$$\mathbb{E} \left[\left(y - \hat{f}(x) \right)^2 \right] = \left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2 + \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right] + \sigma_{\epsilon}^2$$

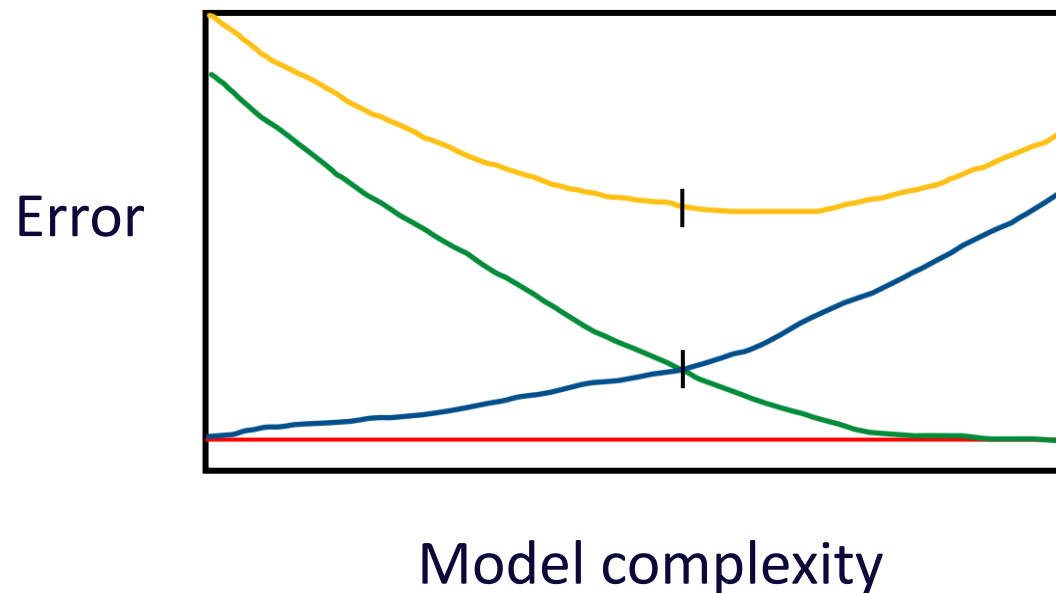
Total error = bias squared + variance + irreducible error



Bias-Variance Tradeoff

$$\mathbb{E} \left[\left(y - \hat{f}(x) \right)^2 \right] = \left(\mathbb{E}[\hat{f}(x)] - f(x) \right)^2 + \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right] + \sigma_{\epsilon}^2$$

Total error = bias squared + variance + irreducible error



Bias-Variance Tradeoff and Dataset Size

- As dataset size goes up, the variance will lower (less overfitting)
- Example: a degree 9 polynomial on 9 datapoints will be overfit
 - But on 10,000 datapoints, much less likely to overfit!
- Bias is not impacted by the dataset size, as bias averages across several sampled datasets
- Takeaway: this is one reason why deep learning needs huge datasets: to avoid overfitting with many trainable parameters!

sklearn demo

- We'll use the sklearn library for regression in Assignment 1
- Two methods of Linear Regression
 - `from sklearn.linear_model import LinearRegression`
 - `from sklearn.linear_model import SGDRegressor`
- LinearRegression: uses Ordinary Least Squares
- SGDRegressor: uses *stochastic* gradient descent
 - Stochastic gradient descent only calculates the gradients w.r.t a random subset of parameters; less accurate, but much faster